9/10/26 work ep class

1. forEach() — Loop Through an Array



2. Example 2: Print marks of every student



3. Example 3: Print items with their index



4. filter() — Select Specific Elements

main.js Run ▶ Output

```
let marks = [45, 82, 33, 91, 67, 28, 75];

let passed = marks.filter((mark) => mark >= 40);
let failed = marks.filter((mark) => mark < 40);

console.log("Passed:", passed);
console.log("Failed:", failed);
```

```
Passed: [ 45, 82, 91, 67, 75 ]
Failed: [ 33, 28 ]
```

5. Example 2: Filter prices below ₹500

main.js Run ▶ Output

```
let prices = [200, 750, 350, 1200, 450, 100];

let result = prices.filter((price) => price < 500);

console.log(result);
```

```
[ 200, 350, 450, 100 ]
```

6. Example 3: Filter even numbers

main.js Run ▶ Output

```
let numbers = [10, 15, 20, 25, 30, 35, 40];

let even = numbers.filter((num) => num % 2 === 0);

console.log(even);
```

```
[ 10, 20, 30, 40 ]
```

7. Example 4: Filter students whose names start with A

main.js Run ▶ Output

```
let students = ["Aman", "Priya", "Ankit", "Rahul",
"Arjun"];

let result = students.filter((name) =>
name.startsWith("A"));

console.log(result);
```

```
[ 'Aman', 'Ankit', 'Arjun' ]
```

8. map() — Transform Every Element

Example 1: Apply a 10% discount



9. Example 2: Convert names to uppercase



10. Example 3: Square every number



11. Example 4: Add 5 bonus marks to every student

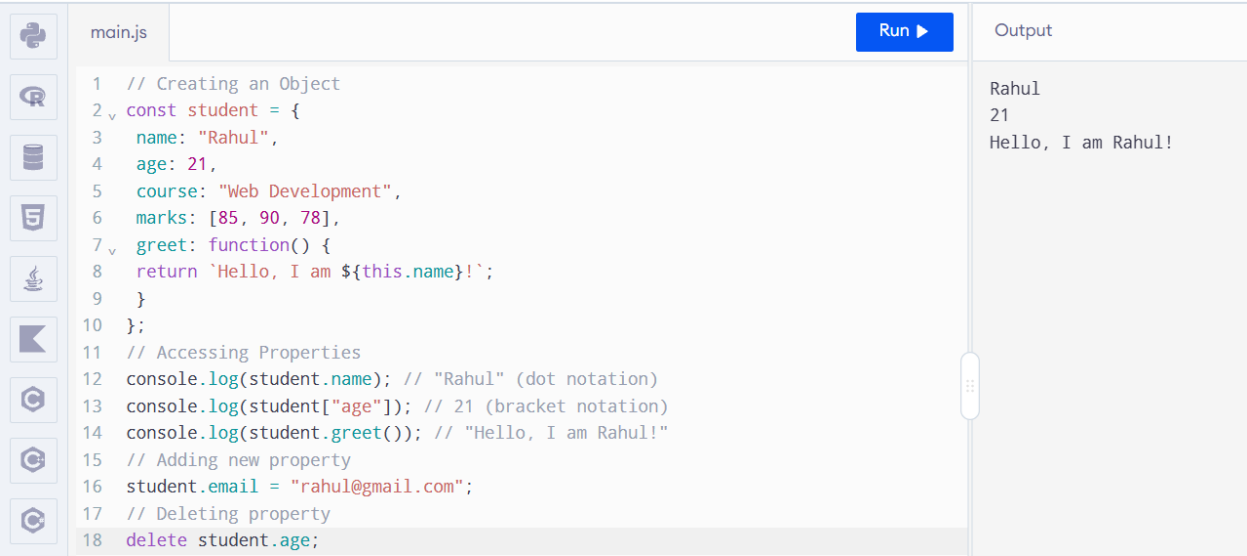


12. reduce() — Combine Array Elements into One Value

Example 1: Calculate total expenses



Object creation & Access



Object destructing



Real world use: When you log into Netflix, your profile data comes as an object: {name: "Rahul", plan: "Premium"}

Classes:

main.js

Run ▶

```
// Define a Class (Blueprint)
class Student {
 constructor(name, age, course) {
  this.name = name;
  this.age = age;
  this.course = course;
 }
 // Method
 introduce() {
  return `Hi, I'm ${this.name}, studying ${this.course}`;
 }
 // Method
 isAdult() {
  return this.age >= 18 ? "Yes" : "No";
 }
}
// Create Objects from Class
const student1 = new Student("Rahul", 21, "Web Dev");
const student2 = new Student("Priya", 19, "Data Science");
console.log(student1.introduce()); // "Hi, I'm Rahul, studying Web Dev"
console.log(student2.isAdult()); // "Yes"
// Inheritance (Class extends another Class):
class Topper extends Student {
 constructor(name, age, course, rank) {
  super(name, age, course); // Call parent constructor
  this.rank = rank;
```

Output

```
Hi, I'm Rahul, studying Web Dev
Yes
Amit is Rank 1!
Hi, I'm Amit, studying Web Dev
```

main.js

Run ▶

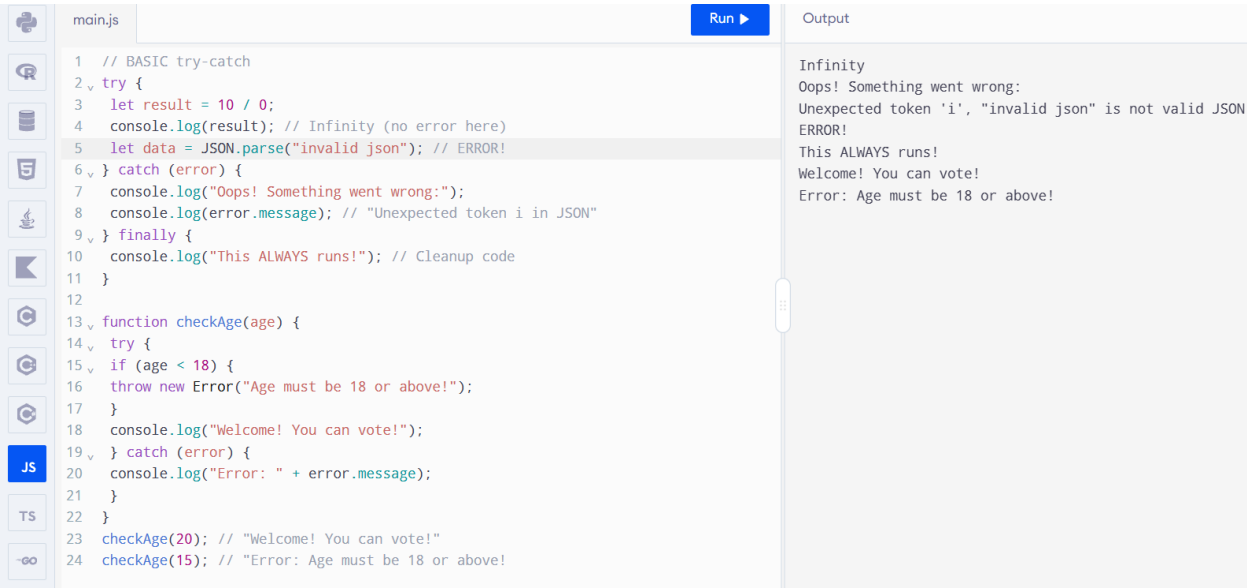
```
 introduce() {
  return `Hi, I'm ${this.name}, studying ${this.course}`;
 }
 // Method
 isAdult() {
  return this.age >= 18 ? "Yes" : "No";
 }
}
// Create Objects from Class
const student1 = new Student("Rahul", 21, "Web Dev");
const student2 = new Student("Priya", 19, "Data Science");
console.log(student1.introduce()); // "Hi, I'm Rahul, studying Web Dev"
console.log(student2.isAdult()); // "Yes"
// Inheritance (Class extends another Class):
class Topper extends Student {
 constructor(name, age, course, rank) {
  super(name, age, course); // Call parent constructor
  this.rank = rank;
 }
showRank() {
 return `${this.name} is Rank ${this.rank}!`;
 }
}
const topper = new Topper("Amit", 22, "Web Dev", 1);
console.log(topper.showRank()); // "Amit is Rank 1!"
console.log(topper.introduce()); // Inherited from Student!
```

Output

```
Hi, I'm Rahul, studying Web Dev
Yes
Amit is Rank 1!
Hi, I'm Amit, studying Web Dev
```

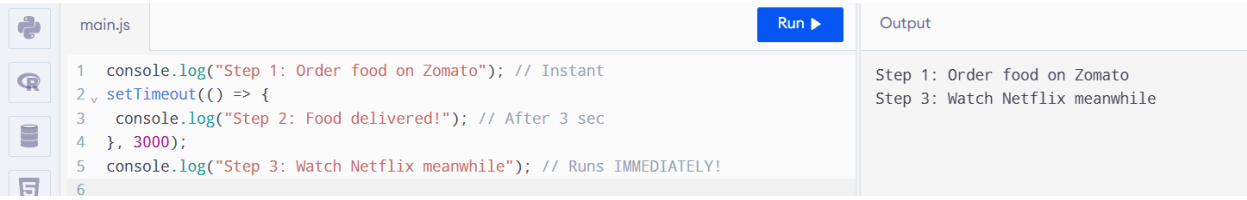
Exception Handling:

Try, catch, finally, throw



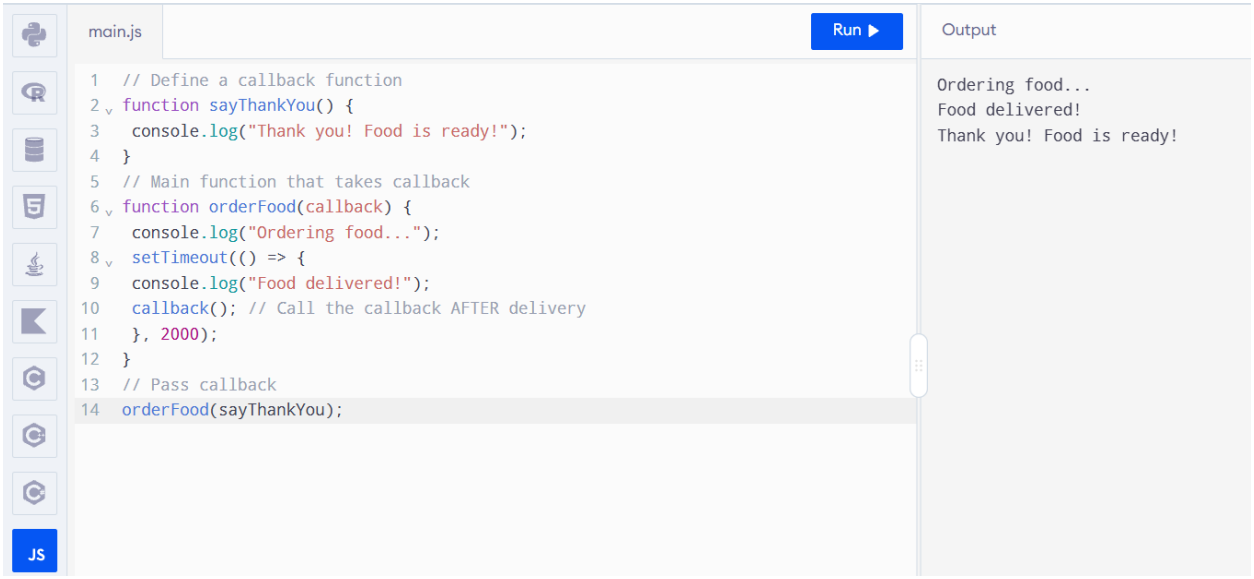
Real-World Use: When you enter a wrong UPI PIN, the app doesn't crash – it shows "Incorrect PIN" using try-catch!

Async Programming:



Real-World Use: When you open Instagram, the feed loads async – you see the skeleton layout first, then photos appear one by one!

Callback function:



Creating & using promises:



Promise.all() - Run Multiple Promises Together

main.js

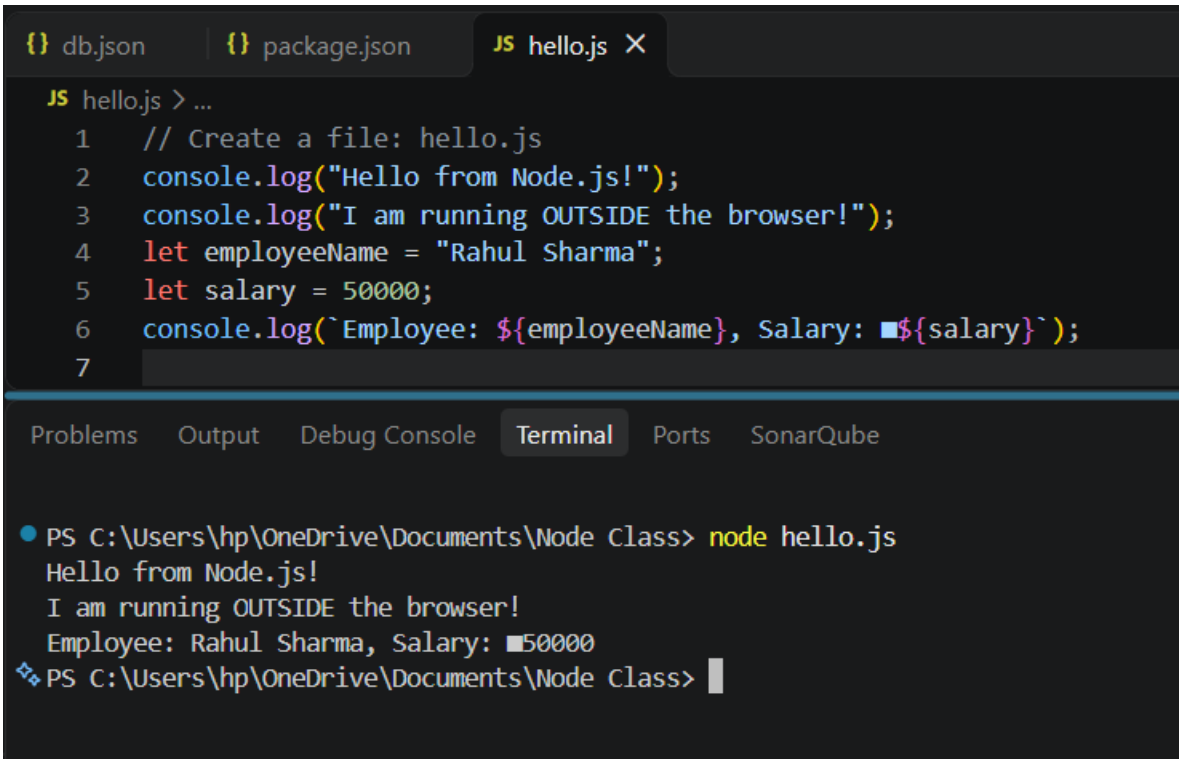
Run ▶

```
const fetchUser = new Promise((resolve) =>
 setTimeout(() => resolve("User Data"), 1000)
);
const fetchPosts = new Promise((resolve) =>
 setTimeout(() => resolve("Posts Data"), 2000)
);
const fetchComments = new Promise((resolve) =>
 setTimeout(() => resolve("Comments Data"), 1500)
);
// Run ALL at once - waits for slowest (2 sec)
Promise.all([fetchUser, fetchPosts, fetchComments])
 .then((results) => {
  console.log(results);
  // ["User Data", "Posts Data", "Comments Data"]
  });
```

Output

```
[ 'User Data', 'Posts Data', 'Comments Data' ]
```

Running JavaScript with Node.js:



Json Server:

```
PS C:\Users\hp\OneDrive\Documents\Node Class> npm i -g json-server

added 45 packages, and audited 46 packages in 18s

14 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\hp\OneDrive\Documents\Node Class> node hello.js
Gurneet
PS C:\Users\hp\OneDrive\Documents\Node Class> node hello
Gurneet
PS C:\Users\hp\OneDrive\Documents\Node Class> npm start

> node-class@1.0.0 start
> node hello.js

Gurneet
```

Filtering:



Payroll Use Case:

✓ All employees in Finance → ?department=Finance

✓ All active employees → ?isActive=true

✓ Female employees in HR → ?department=HR&gender=Female